CCA-F Practice Session

28 October 2026 · Round 1 diagnostic (15 questions), explanations, lessons, and Round 2 drill

Round 1 results: 14 / 15 (93%)

8 answers were marked as guesses, and 7 of those were correct. Counting only answers you were sure of *and* got right, the score is 6 / 15. Q3 and Q15 were not marked and are treated as sure.

Domain	Exam weight	Correct	Sure and correct	Status
D1 Agentic Architecture & Orchestration	27%	3/4	1/4	**Weakest**
D4 Prompt Engineering & Structured Output	20%	3/3	0/3	**Weak, all guessed**
D3 Claude Code Configuration & Workflows	20%	3/3	1/3	Shaky
D2 Tool Design & MCP Integration	18%	3/3	3/3	Strong
D5 Context Management & Reliability	15%	2/2	2/2	Strong

> Focus next: **D1** (the most heavily weighted domain) and **D4**. Target 80% or more in every domain, with answers marked sure.

Round 1 questions and explanations

Green = correct answer. Red with strike-through = your answer, where it was wrong.

Q1 Customer Support Resolution Agent · D1 — Correct

The system prompt says: "Always verify the customer's identity with get_customer before calling process_refund." Logs show that in about 3% of conversations the agent refunds without verifying, usually when the customer sounds urgent. What is the most reliable fix?

A) Rewrite the instruction in capitals and add three few-shot examples showing verification before refunds

B) Add a reviewer agent that audits refunds after they're issued and reverses unverified ones

C) Enforce the order in code: block process_refund unless get_customer has succeeded for this customer in the current session

D) Lower the temperature so the agent follows the system prompt more consistently

Your answer: **C** (Sure) · Correct answer: **C**

Why: Prompt instructions are followed most of the time, not every time. A business rule that involves money needs deterministic enforcement: a code-level gate (or hook) that refuses process_refund until verification has succeeded. A reduces the failure rate but never to zero. B finds the problem after the harm is done. D makes behaviour more consistent, not more obedient.

Q2 Customer Support Resolution Agent · D2 — Correct

process_refund returns {"error": "Operation failed"} for every failure. When an account is closed, so policy doesn't allow a refund, the agent retries five times and then tells the customer "a temporary glitch occurred." What should change?

A) Add "never retry more than once" to the system prompt

B) Return structured errors that state the category (for example business rule or temporary), whether it's worth retrying, and a short explanation the agent can pass on

C) Have the tool raise an exception so the agent loop stops immediately

D) Return an empty success response so the agent moves on

Your answer: **B** (Sure) · Correct answer: **B**

Why: The agent can only make a good decision if the error tells it what happened. A structured error (category, isRetryable, message) lets it skip pointless retries and explain the real reason to the customer. A still gives the agent no way to tell error types apart. C ends the conversation badly. D hides the failure, so the agent tells the customer something false.

Q3 Customer Support Resolution Agent · D5 — Correct

After a long conversation, the history is summarized to save space. Afterwards the agent refers to "your recent order" and asks the customer for the refund amount they already gave. Which design prevents this?

A) Keep a separate "case facts" block (order IDs, amounts, dates, promised actions) that is never summarized and is included in every request

B) Summarize more often, so each summary covers less history

C) Switch to a model with a larger context window

D) Ask the customer to restate key details whenever a summary happens

Your answer: **A** (Not marked) · Correct answer: **A**

Why: Summaries keep the gist but lose exact values such as IDs, amounts and dates. Keeping those transactional facts in a persistent block that is sent with every request protects them. B loses detail faster, because each summary paraphrases the last one. C only delays the problem. D annoys the customer.

Q4 Customer Support Resolution Agent · D2 — Correct

The agent has two tools: get_customer ("Gets customer information") and lookup_order ("Gets order information"). When users give an order number like ORD-48213, the agent often calls get_customer. What should you try first?

A) Expand both descriptions: what each returns, what input format it accepts (for example ORD-12345 for orders), and when to use it instead of the other tool

B) Merge the two tools into one get_info tool

C) Add a separate classifier that picks the tool before the agent runs

D) Add ten few-shot examples of tool choice to the system prompt

Your answer: **A** (Sure) · Correct answer: **A**

Why: Tool descriptions are the main signal the model uses to pick a tool, so improving them is the cheapest, most direct first fix. B blurs the tools further. C adds a whole component, which is over-engineering for a first step. D can help, but adds tokens to every request and leaves the root cause, vague descriptions, in place.

Q5 Multi-Agent Research System · D1 — Incorrect

The user tells the coordinator: "Only peer-reviewed sources from 2023 onward." The coordinator gives the search subagent the task "Find studies on remote work and productivity." The results include blog posts and 2018 papers. What is the root cause?

A) The subagent never sees the coordinator's conversation, and the constraints weren't included in its task

B) The search subagent needs a more capable model

C) Subagents should share one conversation history so constraints carry over automatically

~~**D)** The search tool's description is too vague~~

Your answer: **D** (Guessed) · Correct answer: **A**

Why: A subagent starts with an empty context. It knows only what is written in the task it receives. The constraints were given to the coordinator but never passed on. Fix: write the constraints (and any earlier results it needs) into the subagent's task. B: a stronger model still can't follow rules it never saw. C: subagents don't share history automatically, and copying everything would waste tokens and bury the important part. D: a vague description causes the wrong tool to be picked; here the right tool was used, it just had no constraints. Exam tell: when a user requirement is missing from a subagent's output, check what the coordinator passed down.

Q6 Multi-Agent Research System · D1 — Correct · guessed

A report on "the impact of AI on creative industries" covers only visual art. Logs show the coordinator created three subtasks: AI in digital art, AI in graphic design, and AI in photography. Each subagent did its task well, and the synthesis agent used all the findings. Where is the problem?

A) The synthesis agent's prompt

B) The search subagent's query generation

C) The synthesis agent's context window

D) The coordinator's task breakdown, which never covered music, writing or film

Your answer: **D** (Guessed) · Correct answer: **D**

Why: Every worker did its job, yet whole areas are missing, so the gap was created when the coordinator split the task. Correct but incomplete results point to decomposition; wrong results point to a subagent or its tools. A, B and C cannot add topics that were never assigned.

Q7 Multi-Agent Research System · D5 — Correct

One of four web-search subagents times out. What should it return to the coordinator?

A) An empty result list marked as success, so the pipeline keeps moving

B) A structured failure: what it tried, the error type, any partial results, and whether a retry might work, so the coordinator can decide

C) Nothing, because the coordinator should cancel the whole research run

D) Keep retrying until it succeeds, however long that takes

Your answer: **B** (Sure) · Correct answer: **B**

Why: Errors should travel up with enough context for the coordinator to decide: retry, try another source, or note the gap in the report. A is the worst option: a silent failure makes the report look complete when it isn't. C throws away good results from the other subagents. D can hang the whole run.

Q8 Code Generation with Claude Code · D3 — Correct

The team's coding standards are in the tech lead's ~/.claude/CLAUDE.md. A new developer cloned the repo, and Claude ignores the standards for them. What is the best fix?

A) Ask the new developer to copy the file into their own ~/.claude/ folder

B) Move the standards into the project's CLAUDE.md and commit it to the repository

C) Put the standards in CLAUDE.local.md at the repo root

D) Paste the standards at the start of each session

Your answer: **B** (Sure) · Correct answer: **B**

Why: ~/.claude/ is personal to one machine. Team rules belong in the project CLAUDE.md, committed to git, so everyone gets them and changes go through review. A works until the lead updates the file, then copies drift apart. C: CLAUDE.local.md is for personal notes and is not shared. D depends on people remembering.

Q9 Code Generation with Claude Code · D3 — Correct · guessed

Test files (*.test.ts) sit next to source files across 40 folders. You want the testing conventions loaded only when Claude works on test files. What is the best option?

A) Add the conventions to the root CLAUDE.md

B) Add a CLAUDE.md file in each of the 40 folders

C) Create a skill that developers run when writing tests

D) Add a file under .claude/rules/ with a path pattern such as **/*.test.ts

Your answer: **D** (Guessed) · Correct answer: **D**

Why: A rules file with a paths pattern loads only when Claude works on matching files, which suits file types spread across many folders. A loads test rules for every task. B duplicates the same text 40 times and still loads it for non-test files in those folders. C depends on the skill being triggered and is not tied to file paths. Folder CLAUDE.md files suit code grouped in one place; path rules suit file types scattered everywhere.

Q10 Code Generation with Claude Code · D1 — Correct · guessed

Yesterday Claude analyzed a module in a long session. Overnight, a teammate rewrote most of those files. You want to continue today. What is the best approach?

A) Resume yesterday's session and continue, because Claude will notice the changes

B) Start a new session with a short summary of yesterday's conclusions, so Claude reads the current files fresh

C) Resume the session and run /compact first

D) Fork yesterday's session so the original is kept

Your answer: **B** (Guessed) · Correct answer: **B**

Why: A resumed session still holds the old file contents Claude read earlier. When most files changed, that context is misleading, so start fresh and carry over only the conclusions. A trusts stale tool results. C compacts the stale content instead of removing it. D copies the stale context. Rule: few known changes -> resume and say which files changed; most files changed -> new session plus summary; two approaches from one baseline -> fork.

Q11 Claude Code for CI/CD · D3 — Correct · guessed

A CI job runs claude "Review this pull request for security issues". The job hangs until it times out. What fixes it?

A) Add the -p flag so Claude runs non-interactively, prints the result and exits

B) Increase the CI job's timeout

C) Pipe yes into the command to answer prompts

D) Switch to a faster model

Your answer: **A** (Guessed) · Correct answer: **A**

Why: Without -p, Claude starts an interactive session and waits for typed input that never comes in CI. -p (print mode) runs the prompt, prints the result and exits; add --output-format json and --allowedTools for pipelines. B only makes it hang longer. C is a fragile hack. D doesn't change the waiting.

Q12 Claude Code for CI/CD · D4 — Correct · guessed

There are two jobs: a pre-merge check developers wait on before merging, and a nightly technical-debt report. A manager wants both moved to the Message Batches API to cut costs by 50%. What is the best response?

A) Move both, because the savings are worth it

B) Move only the pre-merge check, since it runs most often

C) Move only the nightly report: batch results can take up to 24 hours, which is fine overnight but would block developers

D) Move neither, because batches don't support tool use

Your answer: **C** (Guessed) · Correct answer: **C**

Why: Batches cost about 50% less but have no fast-response guarantee (up to 24 hours). That suits work nobody is waiting on, like an overnight report, and not a check that blocks merging. A and B block developers. D is not a valid reason.

Q13 Structured Data Extraction · D4 — Correct · guessed

You extract invoice data using a forced tool_use schema. Many invoices have no PO number, and for those the model fills in a plausible-looking one. What is the best fix?

A) Add "Do not make up values" to the prompt

B) Check PO numbers with a regex after extraction

C) Set the temperature to 0

D) Make po_number nullable in the schema and instruct the model to return null when it isn't in the document

Your answer: **D** (Guessed) · Correct answer: **D**

Why: If a schema field is required, the model must put something there, so it invents a value. Making the field nullable gives it an honest way to say "not found". A helps a little but the schema still demands a value. B: an invented number in the right format passes a regex. C makes the same invention repeat every time.

Q14 Structured Data Extraction · D4 — Correct · guessed

Extractions pass the JSON schema, but in 8% of invoices the line items don't add up to the stated total. What is the best next step?

A) Retry with the same prompt, since a second attempt is usually different

B) Accept the output, because schema validation already passed

C) Switch to a larger model for all invoices

D) Check the totals in code, and on failure retry with the specific error included ("line items sum to 950, stated total is 1,000")

Your answer: **D** (Guessed) · Correct answer: **D**

Why: A schema check proves the structure is right, not that the values make sense. Add a semantic check in code, and on failure retry with the specific error so the model knows what to fix. If the information isn't in the document, retrying can't help: return null or send it to a person. A repeats the same mistake. B passes bad data on. C costs more and still has no check.

Q15 Developer Productivity and MCP · D2 — Correct

Your team wants everyone on a project to use the same GitHub MCP server, but the access token must not be committed. What is the best setup?

A) Each developer adds the server to their own user-level config

B) Commit the token in .mcp.json and keep the repo private

C) Commit .mcp.json at the project root and reference the token as an environment variable, such as ${GITHUB_TOKEN}

D) Describe the server in CLAUDE.md so Claude connects to it

Your answer: **C** (Not marked) · Correct answer: **C**

Why: A project-level .mcp.json is shared through git, and environment-variable expansion keeps each person's secret out of the file. A leads to inconsistent setups. B puts a secret in git history permanently, private repo or not. D: CLAUDE.md holds instructions, not connection settings.

Lesson 1: D1 Agentic Architecture

1. Subagents don't share memory

A subagent is like a contractor hired for one job: it only knows what's in the brief. A good brief includes:

- the goal and why it matters
- the user's constraints (dates, sources, scope)
- any earlier results it needs, pasted in directly
- the output format you want back

2. The coordinator is the hub

All communication goes coordinator → subagent → coordinator; subagents don't talk to each other directly. This lets the coordinator check quality and coverage, handle failures, and keep one place for logging.

3. Breaking down the task is a design decision

Results correct but **incomplete** → look at how the coordinator split the task. Results **wrong** → look at the subagent or its tools.

4. Independent subtasks can run in parallel

If subtopics don't depend on each other, the coordinator should start those subagents in one response instead of one after another.

5. Resume, fork or start fresh

Situation	Choose
Same work, files mostly unchanged	Resume the session
A few known files changed	Resume and tell Claude which files changed
Most files changed, or the context is muddled	New session plus a summary of key findings
Try two approaches from the same starting point	Fork the session

Lesson 2: D4 Structured Output

1. Schema design prevents made-up answers

- **Nullable fields** for anything that may be missing, so the model can answer "not found".
- **Enums plus an "other" value and a detail field**, so odd documents aren't forced into the wrong category.
- **Required fields only** for things that always exist.

2. Validate, then retry with the specific error

The schema confirms structure; checks in code confirm values (totals, dates, ranges). On failure, retry with that error message included. A retry cannot create information that isn't in the document: then return null or send it to a person.

3. Forcing a tool with tool_choice

Setting	Effect	Use when

"auto"	The model may reply with text instead	Normal conversation
"any"	The model must call one of the tools	Several schemas and the document type is unknown
A specific tool	The model must call that exact tool	You know exactly which schema applies

4. Message Batches API

About 50% cheaper; results within 24 hours with no faster guarantee; each request carries a custom_id. When some fail, resubmit only the failed ones, matched by custom_id.

Round 2: Targeted drill (D1 + D4)

Not yet answered. Answer key and explanations come after you submit. Goal: 7/8 or better, all marked sure.

R1

The coordinator tells the analysis subagent: "Analyze the findings from the search phase." The subagent replies that it has no findings to analyze. What is the best fix?

A) Give the analysis subagent the search tool so it can redo the searches

B) Include the search findings, or the relevant excerpts, directly in the analysis subagent's task

C) Tell the analysis subagent to read the coordinator's conversation history

D) Increase the analysis subagent's maximum number of turns

R2

To save a step, a teammate proposes that the search subagent send its results straight to the synthesis subagent. What is the best response?

A) Approve it, because fewer hops means fewer tokens

B) Approve it, as long as both subagents use the same model

C) Keep routing through the coordinator, so it can check coverage, handle failures and keep one place for observability

D) Merge the search and synthesis subagents into one agent

R3

The coordinator researches four independent subtopics by calling one subagent, waiting for it, then calling the next. The run takes four times longer than needed. What should change?

A) Have the coordinator start all four subagents in a single response so they run in parallel

B) Use a faster model for the subagents

C) Combine the four subtopics into one large subagent task

D) Cache the subagents' system prompts

R4

Claude has finished analyzing a codebase. You want to try two different refactoring approaches, each starting from that same analysis, without one affecting the other. What is the best approach?

A) Continue in the same session and try one approach after the other

B) Start two new sessions and redo the analysis in each

C) Run /compact, then try both approaches

D) Fork the analysis session, once for each approach

R5

Your schema has document_type as an enum: invoice, receipt, contract. Purchase orders keep being labeled invoice. What is the best fix?

A) Add more few-shot examples of invoices

B) Add an other value to the enum, plus a document_type_detail text field

C) Remove the enum and make document_type free text

D) Reject any document with confidence below 0.9

R6

A due date is missing from a scanned invoice. Your pipeline retries the extraction three times with the validation error included, and all three attempts fail. What should happen?

A) Stop retrying this case: return null for the field and send the document to human review

B) Retry up to ten times with a stronger warning

C) Switch to a larger model and retry

D) Fill in the invoice date plus 30 days as a default

R7

You submitted 10,000 documents as a batch, and 120 failed because they were too long. What should you do next?

A) Resubmit the whole batch with a higher max_tokens

B) Process all 10,000 again with synchronous calls

C) Ignore the failures, since 98.8% is good enough

D) Find the failed ones by custom_id, fix them (for example, split long documents into chunks), and resubmit only those

R8

You have three extraction tools, one per document type, and you don't know the type in advance. You must guarantee the model returns structured data instead of text. Which tool_choice do you use?

A) "auto"

B) The invoice tool, forced by name

C) "any"

D) "none", with JSON requested in the prompt